Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense(128, ReLU)} \rightarrow \text{Dense(64, ReLU)} \rightarrow \text{Dense(10, Softmax)}$$

3. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.

- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction
Construct an MLP with
- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training
Compile using
- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation
Compute
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

Python skearn was used to code the entire MLP algorithm for multi-class image classification on Jupyter notebook. This algorithm includes the data loading, preprocessing of images, model building, training, evaluation, automated hyperparameter tuning using RandomizedSearchCV and SciKeras, re-training of the model based on best hyperparameters, and the plotting of the experimental results.

The complete source code is available in the GitHub repo:

`https://github.com/Mohula-Prasath/Deep_learning_lab.git`

1 6.Experimental Results

1.1 Dataset Exploration

The Fashion-MNIST dataset was imported and explored prior to modeling. It is made up of grayscale images of apparel objects of ten distinct classes. The training dataset comprises 60,000 images, while the test dataset comprises 10,000 images. Each image has the size of 28×28 pixels, which was converted to a 784-length feature vector as an input for the neural network model.

- Number of Training Samples: 60,000
- Number of Testing Samples: 10,000
- Image Size: 28×28 pixels
- Number of Features after Flattening: 784
- Number of Classes: 10

1.2 Data Preprocessing

Various preprocessing tasks have been done before training the neural network. Every image was transformed from a 2D image to a 1D vector of 784 pixel intensities. As the pixel intensity was originally ranging between 0 and 255, Min-Max normalization technique was used to normalize the values between 0 and 1. One hot encoding has been used to transform the class labels for the baseline neural network.

The final dataset used for training had the following characteristics:

- Feature Scaling: Min-Max Normalization
- Training Samples: 60,000
- Testing Samples: 10,000
- Validation Split: 20% of the training data
- Input Features: 784
- Output Classes: 10

1.3 Baseline Model Training

The MLP model was built as the initial model for training. The network architecture includes an input layer with 784 neurons and two hidden layers with 128 and 64 neurons respectively. In the hidden layers, the ReLU activation function was applied, while the output layer used the Softmax activation function to classify images into one of the ten classes.

For the training of the model, the Adam optimizer and categorical cross-entropy loss were used for 20 epochs. The batch size was set to 32, and the validation split to 20%.

The baseline network configuration is summarized below:

- Hidden Layers: 2
- Hidden Neurons: 128 and 64
- Activation Function: ReLU
- Output Activation: Softmax
- Optimizer: Adam
- Batch Size: 32
- Number of Epochs: 20

1.4 Hyperparameter Optimization

To optimize the baseline model, randomized search cross-validation was conducted through the SciKeras wrapper. Various hyperparameter values were tested, which include hidden layers, the number of neurons, activation function, optimizer, learning rate, drop-out rate, batch size, and epochs.

The results showed that the following set of hyperparameters produced the highest accuracy score:

- Optimizer: RMSprop
- Learning Rate: 0.001
- Hidden Layers: 3
- Hidden Neurons: 128
- Activation Function: Tanh
- Dropout Rate: 0.2
- Batch Size: 32
- Number of Epochs: 30
- Best Cross-Validation Accuracy: 0.8906

The optimized architecture was then trained using these parameters. Improvement in training accuracy was observed throughout the epochs, whereas the validation accuracy was kept relatively constant, indicating that it generalized well compared to the baseline model.

1.5 Model Evaluation

Both the Baseline Model and Optimized Neural Network Models were tested on Accuracy, Precision, Recall, and F1-score using the unseen test data set.

Baseline Model had an accuracy of 87.39% while the Optimized Model gave an improved accuracy of 88.08%. The same improvements were noted in Precision, Recall, and F1-scores.

The final evaluation metrics are presented below.

- **Baseline Model**
  - Accuracy: 0.8739
  - Precision: 0.8778
  - Recall: 0.8739
  - F1-Score: 0.8740
- **Optimized Model**
  - Accuracy: 0.8808
  - Precision: 0.8818
  - Recall: 0.8808
  - F1-Score: 0.8809

The confusion matrix and classification report also indicated that there were fewer instances of misclassification of clothes belonging to certain categories due to the use of the optimized model. The accuracy and loss curve plots for training and validation indicated that the model had been able to learn without overfitting. In summary,

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

8. Mandatory Plots

1. Sample Images

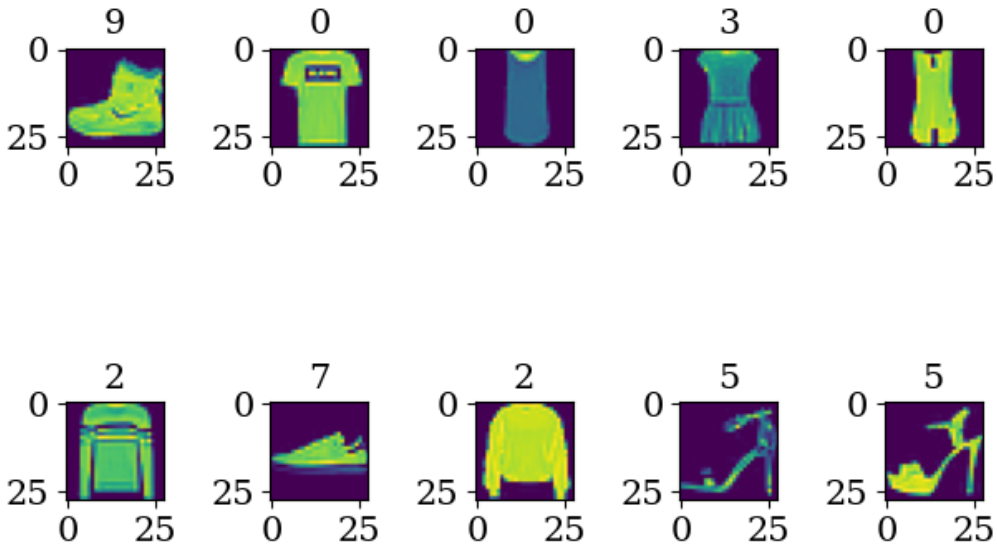

Figure 1: Sample Images from the Fashion-MNIST Dataset

Inference: The above image shows sample images of the fashion-mnist dataset. In the sample we could observe shoe, t-shirt, heels etc

2. Class Distribution

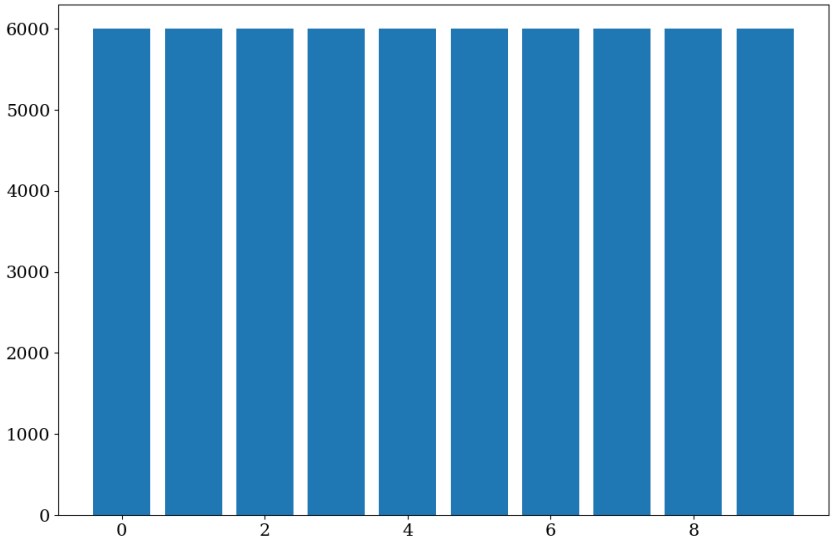

Figure 2: Class Distribution

Inference: In the above image we could clearly see all the 10 images are equally splitted, so we don't have to worry about the class imabalance.

3. Training Accuracy vs Epoch

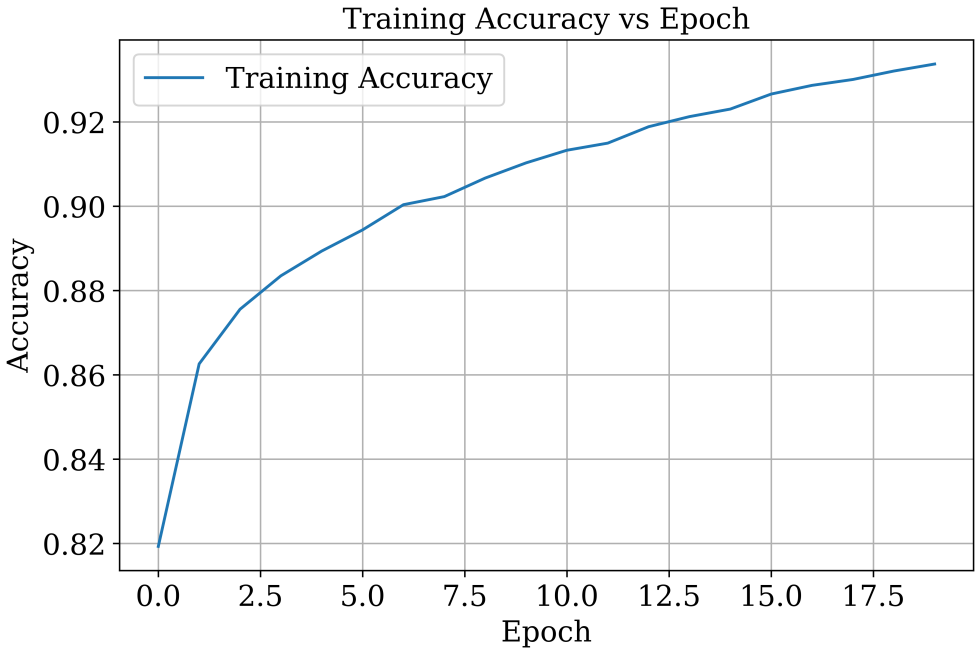

Figure 3: Training Accuracy vs Epoch

Inference: We can see from the above image that the training accuracy increases rapidly and ends above 0.92

4. Validation Accuracy vs Epoch

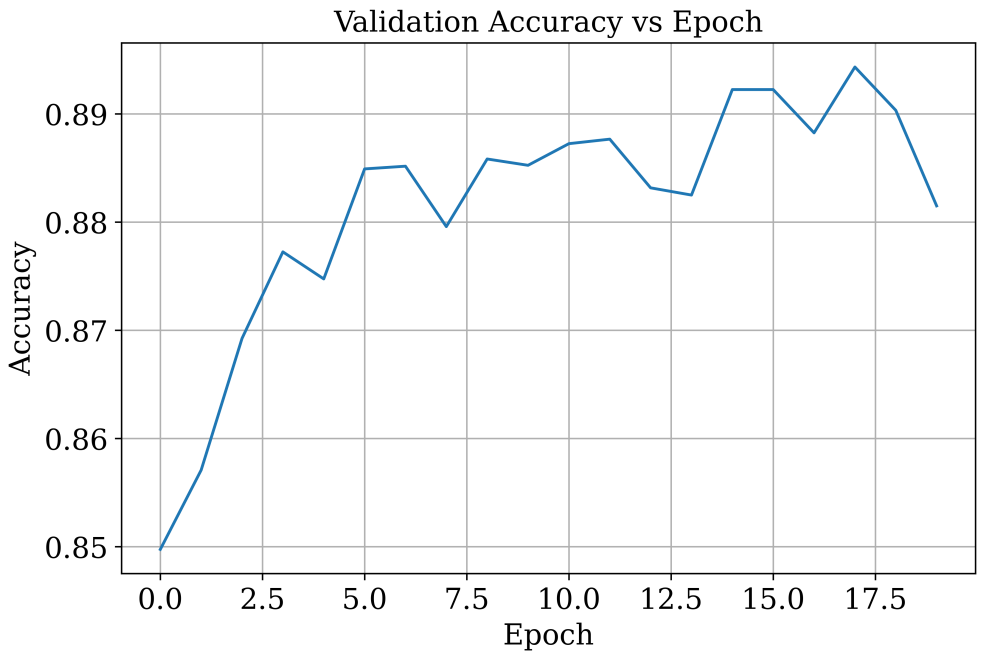

Figure 4: Validation Accuracy vs Epoch

Inference: The validation accuracy initally increases upto 17 epochs but at the end we can see there is a sudden dip

5. Training Loss vs Epoch

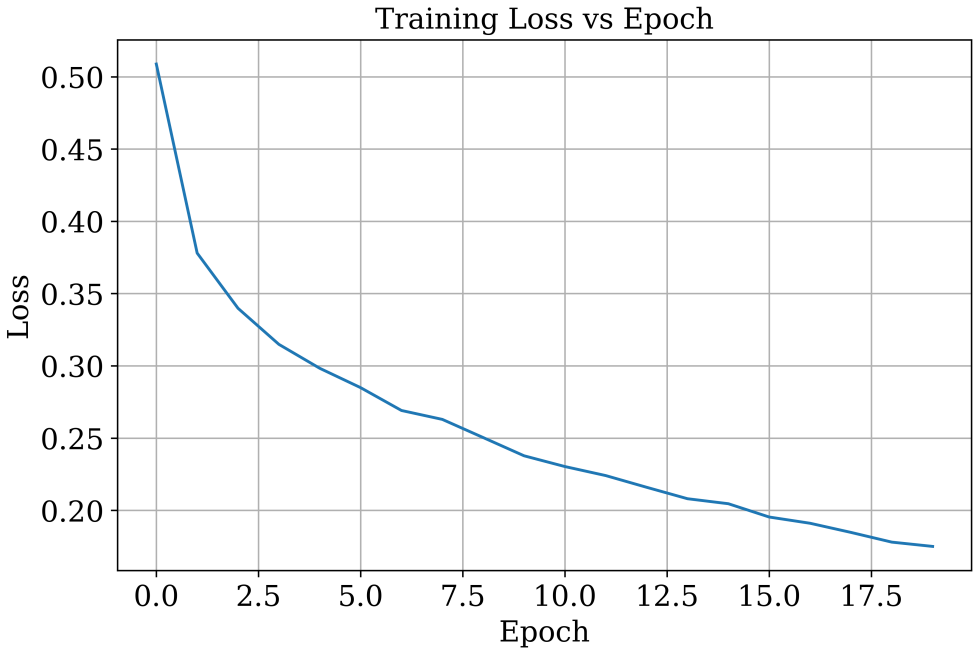

Figure 5: Training Loss vs Epoch

Inference: The loss decreases steeply which indicates that the model is performing well

6. Validation Loss vs Epoch

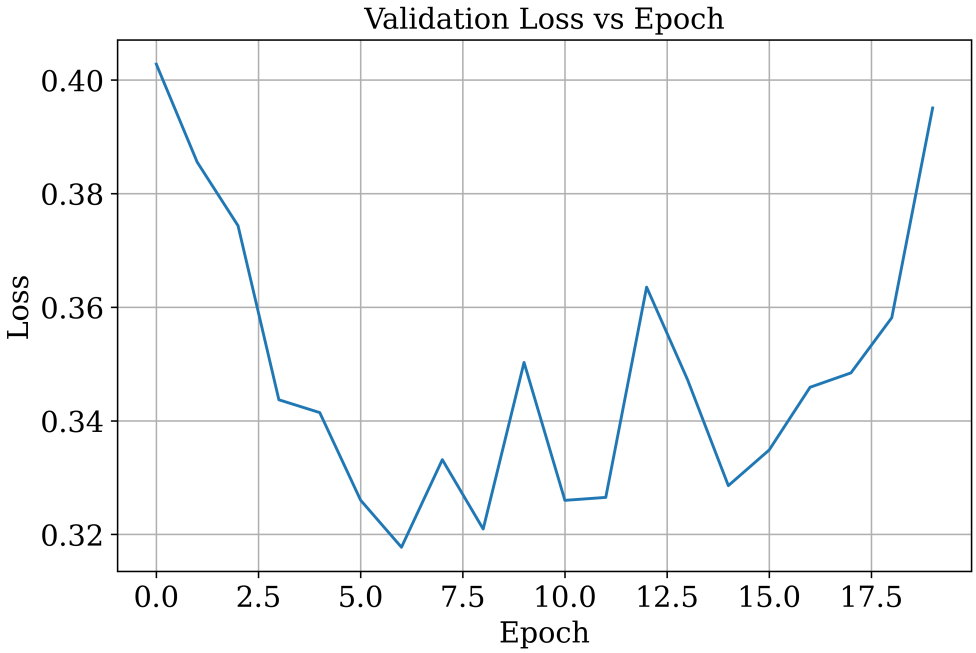

Figure 6: Validation Loss vs Epoch

Inference: From this image we can see there are lot of fluctuation in the validation loss vs epoch graph but at last the loss increases

7. Confusion Matrix

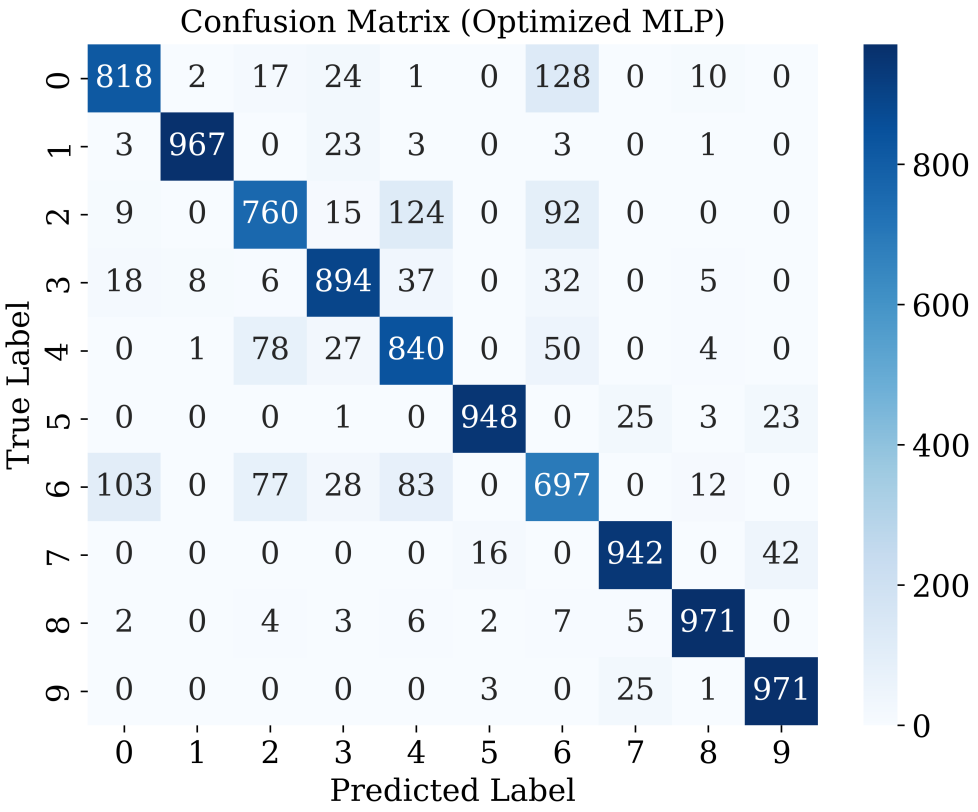

Figure 7: Confusion Matrix of the Optimized MLP

Inference: As we can see from the confusion matrix that most of the predictions were right but there are errors too we can clearly see model faced difficulty in distinguishing 6 and 0.

8. Hyperparameter Search Results

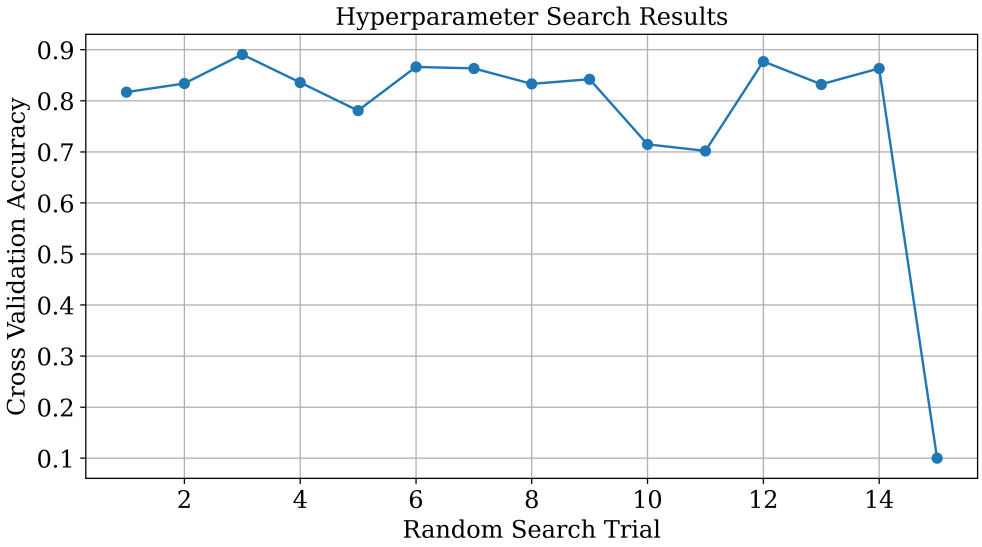

Figure 8: Randomized Hyperparameter Search Results

Inference: From the graph above we can see the last point is too low in accuracy whereas others performed well the lowest accuracy goes upto 0.1

9. Best Model Accuracy Comparison

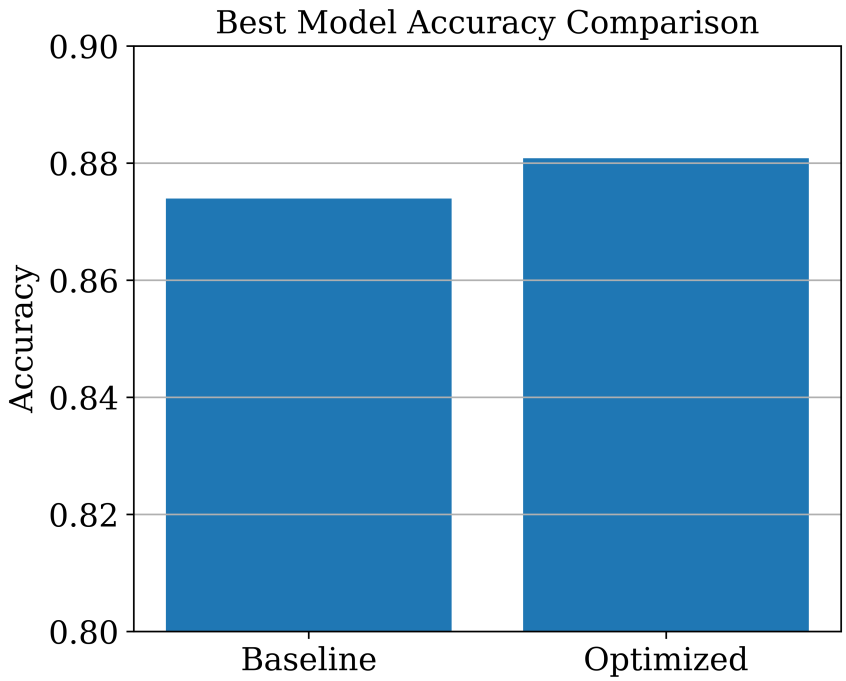

Figure 9: Baseline vs Optimized Model Accuracy

Inference: From the image we can see both the baseline model and the optimised model produced almost same accuracy this is because of less number of trials in RandomizedSearchCV

9. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	rmsprop
Activation Function	tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	0.8906333333333334
Testing Accuracy	0.8808

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.873900	0.878700
Precision	0.877836	0.882449
Recall	0.873900	0.878700
F1-score	0.874008	0.879320
Training Time	130s	150s

10. Discussion

Answer the following:

1. Which optimization method was used?
   The optimization method used was **RandomizedSearchCV**.
2. Which hyperparameters were selected?
   The best hyperparameters selected by **RandomizedSearchCV** were:
   - Optimizer: RMSprop
   - Learning Rate: 0.001
   - Hidden Neurons: 128
   - Hidden Layers: 3
   - Dropout Rate: 0.2
   - Activation Function: tanh
   - Epochs: 30
   - Batch Size: 32
3. Did optimization improve performance?
   Yes, the optimization improved the model's performance.
4. Which hyperparameter had the greatest impact?
   The best model was obtained from the combined effect of all tuned hyperparameters as we have used randomisedCV

The hyperparameter tuning was performed using the following code:

```
RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=15,
    cv=5,
    scoring='accuracy',
    random_state=42
)
```

5. Compare Grid Search and Randomized Search.
   - Grid Search evaluates every possible combination of hyperparameters
   - Randomized Search evaluates only a random subset of combinations
   - Grid Search is computationally expensive whereas Randomized Search is faster
   - Randomized Search is preferred for large hyperparameter search spaces.
6. Recommend the best MLP configuration.
   The recommended MLP configuration obtained from the optimization process is:
   - Optimizer: RMSprop
   - Learning Rate: 0.001
   - Hidden Layers: 3
   - Hidden Neurons: 128
   - Dropout Rate: 0.2
   - Activation Function: tanh
   - Epochs: 30
   - Batch Size: 32

   This configuration achieved a Cross-Validation Accuracy of **89.06%** and a Test Accuracy of **88.08%**.

11. Additional Task: Perceptron Learning Algorithm for XOR Gate

11.1 Objective

To implement the Perceptron Learning Algorithm for the XOR gate using Python, observe the weight updates after each iteration, visualize the decision boundary after selected epochs, and analyze why the perceptron fails to converge for the XOR problem.

11.2 Experimental Procedure

- Create the XOR truth table.
- Initialize the perceptron weights and bias to zero.
- Perform forward propagation using the step activation function.
- Update the weights using the perceptron learning rule whenever a sample is misclassified.
- Plot the decision boundary after every epoch.
- Continue the training for 20 epochs and observe the convergence behaviour.

11.3 Experimental Results

The perceptron was changing its weights all the time it was being trained.. It was different from the OR and AND gates. The perceptron was still getting some things wrong after trying a lot. It just could not make up its mind where to draw the line. The perceptron was trying to figure things out for 20 epochs. It still did not get it right. This shows that a single-layer perceptron is not good at figuring out the XOR

gate. The reason is that the XOR gate data is not simple enough for the perceptron to understand. The perceptron needs data that's linearly separable to work correctly. The XOR gate data is just not, like that. The perceptron cannot correctly classify the XOR gate.

11.4 Plots

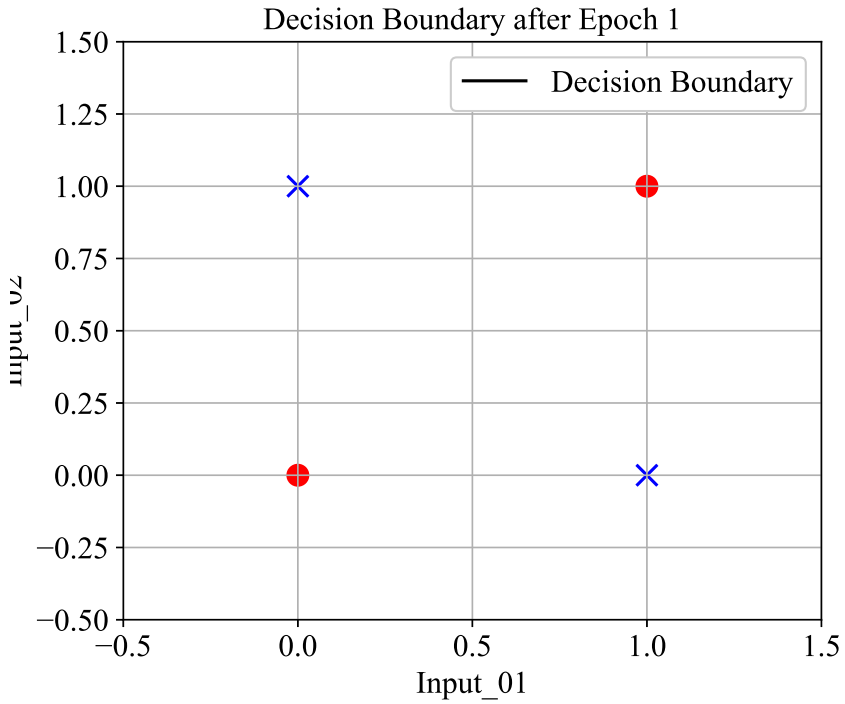

Figure 10: XOR Decision Boundary after Epoch 1

Interpretation:

During the first epoch the perceptron begins adjusting the weights based on the classification errors. The initial separating boundary is unable to distinguish the XOR classes correctly because positive and negative samples are positioned diagonally opposite to one another.

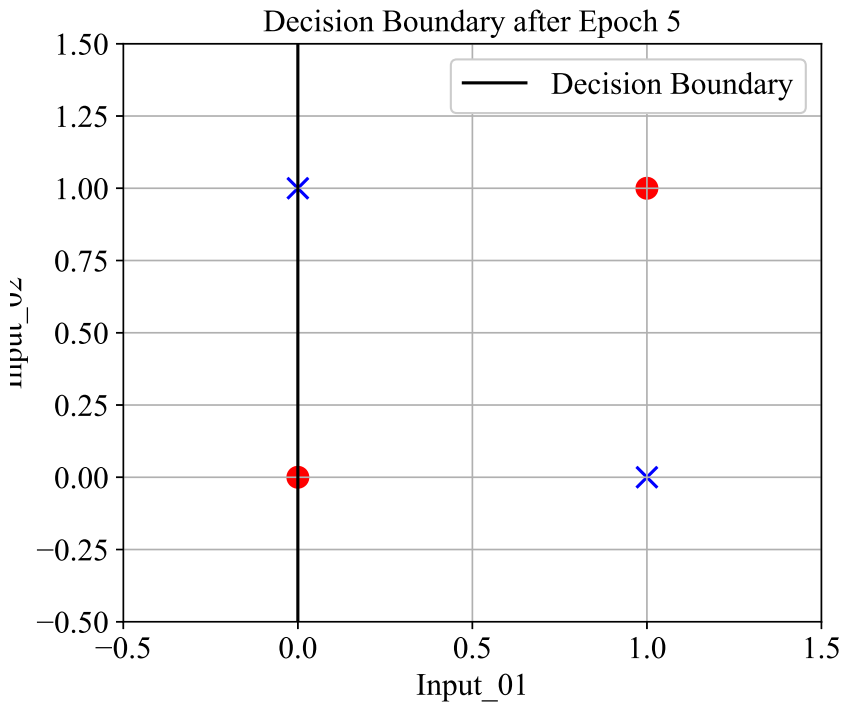

Figure 11: XOR Decision Boundary after Epoch 5

Interpretation:

After several weight updates, the decision boundary changes its orientation but still fails to separate all the data points correctly. Some samples remain misclassified, causing additional weight updates during the next epochs.

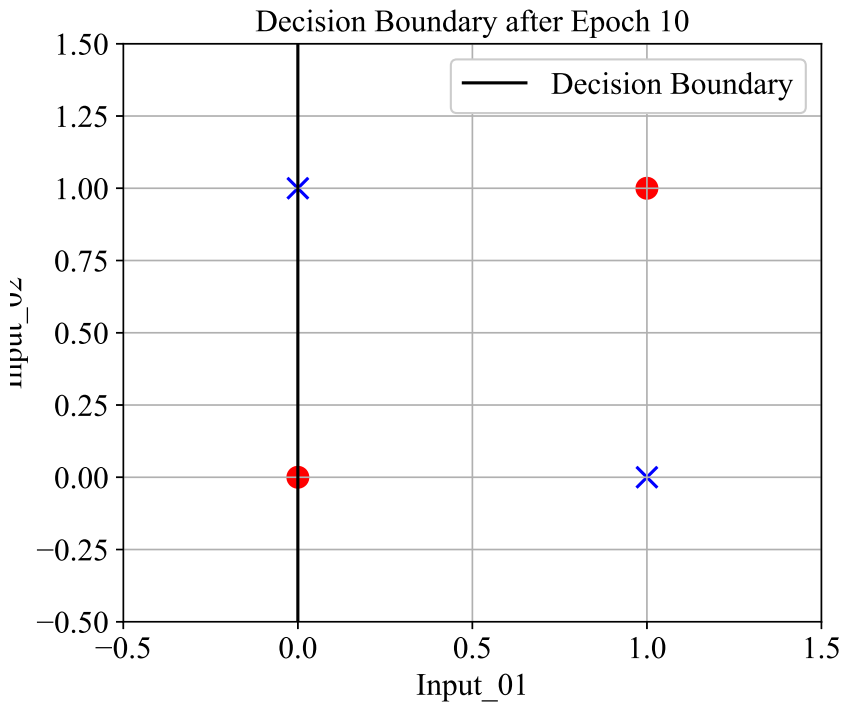

Figure 12: XOR Decision Boundary after Epoch 10

Interpretation:

By the tenth epoch, the perceptron continues modifying its weights, but the decision boundary is still unable to correctly classify every sample simultaneously. The weights oscillate because improving the classification of one sample causes another sample to become misclassified.

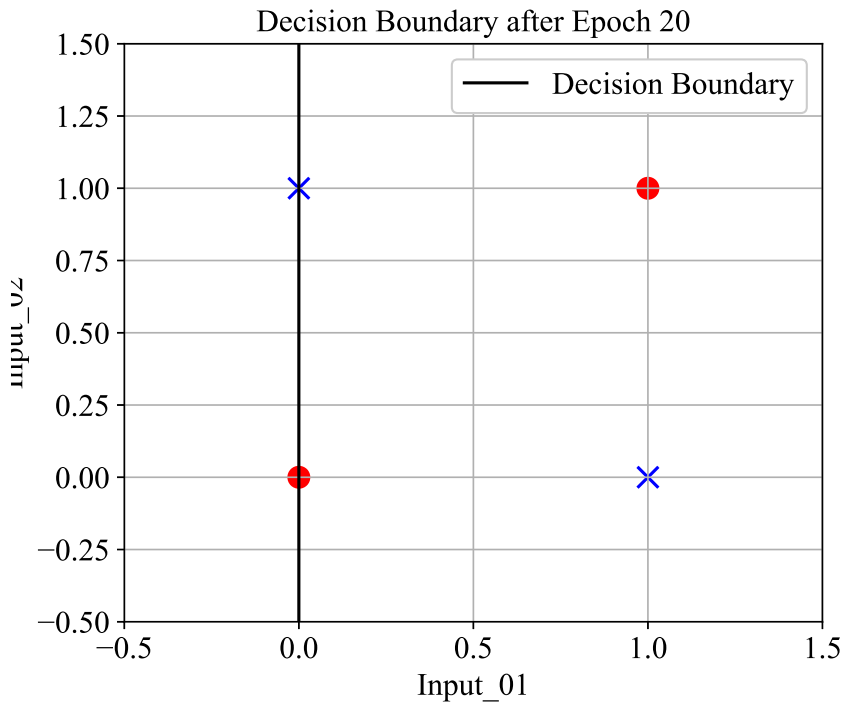

Figure 13: XOR Decision Boundary after Epoch 20

Interpretation:

Even after twenty epochs, the perceptron does not converge to a stable solution. The separating line continues to oscillate because the XOR dataset is not linearly separable. A single linear decision boundary cannot separate the two classes correctly.

11.5 Analysis

The XOR gate is different from the OR gate, the AND gate and the NOT gate. This is because the output of the XOR gate cannot be divided into classes using a straight line. The perceptron learning algorithm needs the training data to be linearly separable. This means the algorithm thinks the data can be separated into classes using a line.

The XOR gate does not work this way. Consequently the weights of the perceptron keep changing during the training process. They do not reach a solution.

This shows why we need -layer neural networks with hidden layers to solve problems like the XOR gate. The XOR gate is a -linearly separable problem. This means it cannot be solved using a straight line. Multi-layer neural networks, with layers can solve this kind of problem.

11. Conclusion

In this experiment, a Multi-Layer Perceptron was successfully built and trained for multi-class classification of images from the Fashion-MNIST dataset. The baseline model, comprising of 2 hidden layers with 128 and 64 neurons respectively, yielded a test accuracy of 87.39%. However, hyperparameter optimization carried out by RandomizedSearchCV using the SciKeras wrapper led to discovery of a better combination – 3 hidden layers, 128 neurons, tanh activation function, RMSprop optimizer, and 0.2 dropout probability. This yielded a cross-validation accuracy of 89.06% and a test accuracy of 88.08%.

This shows that systematic hyperparameter tuning could lead to marginal but consistent improvements in terms of accuracy, precision, recall, and F1-score. The training and validation curves indicate that the optimized model generalizes quite well without significant overfitting.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.